

WEEK-2: Mini UI System with Core Concepts

Aim:

To design a mini-UI system demonstrating component-based architecture and unidirectional data flow.

Description:

A startup is developing an internal dashboard for monitoring employee performance, but the application suffers from inconsistent UI behavior and difficulty in debugging. Changes in one part of the interface unintentionally affect other parts due to improper data flow and lack of structure. The absence of a clear component-based architecture makes the system hard to scale and maintain. Additionally, direct mutation of data leads to unpredictable UI updates. To solve this, students are required to design and implement a mini UI system using JavaScript that follows key architectural principles such as component-based design, unidirectional data flow, and immutability. The system should also include a basic reconciliation mechanism to efficiently update only the modified parts of the UI, ensuring predictable and maintainable behavior.

Modules:

- Component-based design
- State management
- Unidirectional data flow
- Reconciliation logic
- Immutability

Tasks:

- Create reusable UI components using functions
- Implement a central state object
- Pass data between components (one-way flow)
- Update UI only when state changes
- Demonstrate immutability using spread operator

PROJECT STRUCTURE

Week2/

```
├── index.html
├── style.css
├── state.js
├── renderer.js
├── app.js
├── components/
│   ├── EmployeeList.js
│   └── Controls.js
```

Programs:

1. index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Employee Dashboard</title>
  <link rel="stylesheet" href="style.css">
```

```

</head>
<body>

<div class="container">
  <h1 class="title">Employee Performance Dashboard</h1>

  <div class="btn-group">
    <button onclick="addEmployee()">✚ Add Employee</button>
    <button onclick="updateScore()">↑ Update Score</button>
    <button onclick="downloadPDF()">📄 Download PDF</button>
  </div>

  <div id="app"></div>
</div>

<script type="module" src="app.js"></script>
<!-- jsPDF -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js"></script>

<!-- AutoTable plugin -->
<script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf-autotable/3.5.23/jspdf.plugin.autotable.min.js"></script>
</body>
</html>

```

2. style.css

```

body {
  font-family: Arial;
  background: #f4f4f4;
}

.container {
  width: 70%;
  margin: 40px auto;
  text-align: center;
}

.title {
  font-size: 28px;
}

button {
  padding: 10px 15px;
  margin: 5px;
  border-radius: 20px;
  border: none;
  background: green;
  color: white;
  cursor: pointer;
}

```

```

button:hover {
  background: darkgreen;
}

.search-box {
  width: 60%;
  padding: 10px;
  margin-bottom: 15px;
  border-radius: 20px;
}

table {
  width: 100%;
  background: white;
  border-collapse: collapse;
}

th {
  background: blue;
  color: white;
  padding: 10px;
}

td {
  padding: 10px;
  border-bottom: 1px solid #ddd;
}

```

3. state.js

```

export let state = {
  employees: [
    { id: 1, name: "John", score: 70, present: true },
    { id: 2, name: "Sara", score: 85, present: false }
  ]
};

let listeners = [];

export function subscribe(listener) {
  listeners.push(listener);
}

export function setState(newState) {
  state = { ...state, ...newState }; // immutability
  listeners.forEach(l => l());
}

```

4. renderer.js

```

let previousVDOM = "";

export function render(vdom, container) {

  if (vdom !== previousVDOM) {
    container.innerHTML = vdom;
    previousVDOM = vdom;
    console.log("UI Updated");
  } else {
    console.log("No Change");
  }
}
}

5. app.js
import { state, subscribe } from "./state.js";
import { render } from "./renderer.js";
import { EmployeeList } from "./components/EmployeeList.js";
import "./components/Controls.js";

```

```

const root = document.getElementById("app");

```

```

function App() {
  return `
    ${EmployeeList(state.employees)}
  `;
}

```

```

function updateUI() {
  render(App(), root);
}

```

```

subscribe(updateUI);
updateUI();

```

```

6. components/EmployeeList.js
export function EmployeeList(employees) {

```

```

  return `
    <input
      type="text"
      placeholder="Search Employee..."
      onkeyup="searchEmployee(this.value)"
      class="search-box"
    />

    <table>
      <thead>
        <tr>
          <th>ID</th>
          <th>Name</th>

```

```

        <th>Score</th>
        <th>Attendance</th>
        <th>Actions</th>
    </tr>
</thead>

<tbody>
    ${employees.map(emp => `
        <tr>
            <td>${emp.id}</td>
            <td>${emp.name}</td>
            <td>${emp.score}</td>
            <td>
                <button onclick="toggleAttendance(${emp.id})">
                    ${emp.present ? "Present" : "Absent"}
                </button>
            </td>
            <td>
                <button onclick="editName(${emp.id})">Edit Name</button>
                <button onclick="editScore(${emp.id})">Edit Score</button>
                <button onclick="deleteEmployee(${emp.id})">Delete</button>
            </td>
        </tr>
    `).join("")}
</tbody>
</table>
`;
}

```

7. components/Controls.js

```

import { state, setState } from "../state.js";

// Add Employee
window.addEmployee = function () {
    const name = prompt("Enter Name:");

    if (!name) return;

    const newEmp = {
        id: state.employees.length + 1,
        name,
        score: 50,
        present: true
    };

    setState({
        employees: [...state.employees, newEmp]
    });
};

```

```

// Update Score
// Manual Update Score
window.updateScore = function () {

    const id = Number(prompt("Enter Employee ID:"));
    const newScore = prompt("Enter New Score:");

    if (!id || newScore === null) return;

    const updated = state.employees.map(emp =>
        emp.id === id
        ? { ...emp, score: Number(newScore) }
        : emp
    );

    setState({ employees: updated });
};

// Edit
window.editEmployee = function (id) {
    const name = prompt("Enter new name:");

    const updated = state.employees.map(emp =>
        emp.id === id
        ? { ...emp, name: name || emp.name }
        : emp
    );

    setState({ employees: updated });
};

// Attendance
window.toggleAttendance = function (id) {
    const updated = state.employees.map(emp =>
        emp.id === id
        ? { ...emp, present: !emp.present }
        : emp
    );

    setState({ employees: updated });
};

// Search
window.searchEmployee = function (query) {

    query = query.toLowerCase();

    if (query === "") {
        // restore original data
        setState({ employees: [...state.allEmployees] });
    }
};

```

```

    return;
  }

  const filtered = state.allEmployees.filter(emp =>
    emp.name.toLowerCase().includes(query)
  );

  setState({ employees: filtered });
};

window.deleteEmployee = function (id) {

  const updatedEmployees = state.employees.filter(emp => emp.id !== id);

  setState({
    employees: updatedEmployees
  });
};

window.downloadPDF = function () {

  const { jsPDF } = window.jspdf;
  const doc = new jsPDF();

  // Title
  doc.text("Employee Report", 14, 10);

  // Table Data
  const tableData = state.employees.map(emp => [
    emp.id,
    emp.name,
    emp.score,
    emp.present ? "Present" : "Absent"
  ]);

  // Generate Table
  doc.autoTable({
    head: [["ID", "Name", "Score", "Attendance"]],
    body: tableData
  });

  // Save PDF
  doc.save("employees.pdf");
};

```

Outputs:

Employee Performance Dashboard

+ Add Employee

↑ Update Score

Search Employee...

ID	Name	Score	Attendance	Actions	
1	Subba	70	Present	Edit	Delete
2	Rayudu	85	Absent	Edit	Delete

127.0.0.1:5500/index.html

Software Testing H... automation testing jeni

127.0.0.1:5500 says

Enter Name:

Nani

OK

Cancel

Search Employee...

ID	Name	Score	Attendance	Actions	
1	Subba	70	Present	Edit	Delete
2	Rayudu	85	Absent	Edit	Delete

Employee Performance Dashboard

+ Add Employee

↑ Update Score

Search Employee...

ID	Name	Score	Attendance	Actions	
1	Subba	70	Present	Edit	Delete
2	Rayudu	50	Present	Edit	Delete
3	Nani	50	Present	Edit	Delete

l:5500/index.html

e Testing H...

automation testing

jenl

127.0.0.1:5500 says

Enter Employee ID:

3

OK

Cancel

Search Employee...

ID	Name	Score	Attendance	Actions	
1	Subba	70	Present	Edit	Delete
2	Rayudu	50	Present	Edit	Delete
3	Nani	50	Present	Edit	Delete

5500/index.html

Testing H... automation testing jeni Manuscript List

127.0.0.1:5500 says

Enter New Score:

98

OK Cancel

Search Employee...

ID	Name	Score	Attendance	Actions
1	Subba	70	Present	Edit Delete
2	Rayudu	50	Present	Edit Delete
3	Nani	50	Present	Edit Delete

Employee Performance Dashboard

+ Add Employee + Update Score

Search Employee...

ID	Name	Score	Attendance	Actions
1	Subba	70	Present	Edit Delete
2	Rayudu	50	Present	Edit Delete
3	Nani	98	Present	Edit Delete

Delete Id 3

Employee Performance Dashboard

[+ Add Employee](#) [↑ Update Score](#)

Search Employee...

ID	Name	Score	Attendance	Actions
1	Subba	70	Present	Edit Delete
2	Rayudu	50	Present	Edit Delete

Employee Performance Dashboard

[+ Add Employee](#) [↑ Update Score](#) [Download PDF](#)

Search Employee...

ID	Name	Score	Attendance	Actions
1	Subba	70	Present	Edit Delete
2	Rayudu	85	Absent	Edit Delete



employees.pdf

Pdf file

employees.pdf - Adobe Reader

File Edit View Window Help

Open

1 / 1 141%

Employee Report

ID	Name	Score	Attendance
1	Subba	70	Present
2	Rayudu	85	Absent

Results: Thus, To design a mini UI system demonstrating component-based architecture and unidirectional data flow

Viva Q &A:

1. What is the role of setState() in your program?

setState() is used to update the central state object in state.js. It creates a new updated state using the spread operator and then notifies all subscribed functions to re-render the UI. This ensures that the UI automatically reflects the latest data.

2. How is immutability implemented in your code?

Immutability is implemented using the spread operator (...). Instead of modifying the existing employee object, a new object is created when updating data.

Example:

```
{ ...emp, score: emp.score + 5 }
```

This avoids direct mutation and ensures predictable updates.

3. How does the delete operation work in your program?

The delete operation uses the filter() method to remove an employee from the array. It creates a new array excluding the selected employee and updates the state using setState().

Example:

```
state.employees.filter(emp => emp.id !== id)
```

4. How does the search functionality work?

The search function takes user input and filters the employee list based on the name using the includes() method. It compares the search query with employee names in lowercase and updates the UI with matching results.

5. How is the UI updated when data changes?

Whenever the state is updated using setState(), all subscribed functions are triggered. The render() function compares the new UI (virtual DOM string) with the previous one and updates the actual DOM only if changes are detected.

NOTE: Get Project Zip Folder From <https://github.com/subbusir/FEDF/tree/main/week2>